

Day 6 复习总表：JavaScript 函数与数组

Step 项	完成事项	对应代码 / 指令	代码详解
Step 2 函数声明 与调用	定义并调 用一个简 单函数	<pre>(js) function greet() { console.log("Hello, world!"); } greet();</pre>	• <code>function</code> 函数名 <code>() {...}</code> 声明函 数, <code>greet</code> 是函数名 • <code>greet();</code> 调用函 数, 才会执行函数体 • 函数不调用, 函数体 永远不会运行
Step 3 参数与返 回值	传参、用 <code>return</code> 返 回结果	<pre>(js) function greetUser(name) { console.log("Hello, " + name + "!"); } function add(a, b) { return a + b; } let result = add(5, 3);</pre>	• <code>(name)</code>/<code>(a)</code>/<code>(b)</code> 是参 数, 调用时传入具体值 • <code>return</code> 把结果返回 给调用者, 并且终止函 数执行 • <code>let result = add(5, 3)</code> 把返回值 存进变量, 供后续使用
Step 4 箭头函数	用 ES6 简 写重写同 样的函数	<pre>(js) const multiplyShort = (a, b) => a * b; const square = x => x * x; const sayHi = () => "Hi!";</pre>	• <code>=></code> 箭头函数, 替代 <code>function</code> 关键字 • 函数体只有一行表达 式时可省略 <code>{}</code> 和 <code>return</code> (隐式返 回) • 只有一个参数时括号 可省略; 没有参数时括 号不能省略
Step 5 数组基础	创建、访 问、修 改、遍历 数组	<pre>(js) let fruits = ["Apple", "Banana", "Cherry"]; fruits[1] = "Blueberry"; for (let fruit of fruits) { ... }</pre>	• <code>[]</code> 创建数组, 元素 用逗号分隔, 索引从 0 开始 • <code>.length</code> 获取数组 长度 • <code>for...of</code> 直接拿 到每个元素本身, 比传 统 <code>for</code> 循环更简洁
Step 6 数组方法	<code>push</code> / <code>map</code> /	<pre>(js) numbers.push(4); let doubled = numbers.map(num => num * 2);</pre>	• <code>push()</code> 向末尾添加 元素, 会修改原数组 • <code>map()</code> 对每个元素

Step 项	完成事项	对应代码 / 指令	代码详解
	filter 处理数据	<pre>2); let evenNumbers = numbers.filter(num => num % 2 === 0);</pre>	执行函数，返回等长的新数组，不改原数组 <code>filter()</code> 筛选满足条件的元素，返回可能更短的新数组，也不改原数组
Step 7 综合练习：数据驱动渲染	用对象数组 + map 动态生成项目卡片	<pre>js const projects = [{ title: "Project 1", description: "..."}]; const projectCardsHTML = projects.map(project => return +"<article><h3>\${project.title}</h3> <p>\${project.description}</p></article>"+; }).join(""); document.getElementById("project- container").innerHTML = projectCardsHTML;</pre>	- 数组里存对象，每个对象代表一个项目的数据 <code>map()</code> 把每个数据对象转换成一段 HTML 字符串 - 模板字符串（反引号）+ <code>\${}</code> 把变量嵌入 HTML 里 <code>.join("")</code> 把字符串数组拼成一整段字符串 <code>innerHTML</code> 把拼好的 HTML 插入页面容器
Step 8 Git 提交	保存本日函数+数组+动态渲染代码	<pre>bash git add index.html script.js git commit -m "feat: add functions and arrays, dynamic project rendering"</pre>	一次提交打包"函数与数组学习 + 项目卡片改为数据驱动"这个完整功能
Step 9 Code Review + 自评	通过问答检验函数/数组核心理解	考察问题： <code>return</code> 的作用，不写会怎样？ <code>map()</code> 和 <code>filter()</code> 的区别？ "数据 → 函数 → UI" 有什么好处？	<code>return</code> 把结果返回给调用者并终止函数；不写则函数返回 <code>undefined</code> <code>map()</code> 对所有元素逐个转换，长度不变；<code>filter()</code> 筛选，长度可能变短 - 结构化、易扩展——加第 4 个项目只需要在数组里加一条数据，不用碰 HTML

本日核心概念速查

概念	一句话记忆
声明式函数 vs 箭头函数	前者可"提升"先调用后定义；后者更短，但要先定义
<code>push</code> 改原数组， <code>map</code> / <code>filter</code> 不改	这是最容易搞混的一对概念，务必记牢
模板字符串 <code>`\${}`</code>	变量直接嵌进字符串，替代繁琐的 <code>+</code> 拼接
<code>.join("")</code>	把 <code>map</code> 生成的字符串数组，合并成一整段可插入页面的字符串
数据驱动渲染	页面内容由数组决定，改数据不用改 HTML，是 React 思想的雏形